

Python, NumPy, Pandas and a First Honest Evaluation

Does the data contain signal — and how do we know a model has learned it?

Module	Machine Learning	Chapter	1 — Introduction to Machine Learning
Level	3rd-year Software Engineering	Instructor	Professor Nizar Omheni
Duration	90 min in class (+ pre-lab installation)	Format	Pairs, one submission per pair
Institution	Institut Supérieur d'Informatique et de Mathématiques de Monastir (ISIMM), Université de Monastir		

1 Scenario and practical objective

Hypothetical scenario (for teaching purposes). A botanical garden's herbarium team records four measurements on iris flowers: sepal length, sepal width, petal length and petal width (in cm). Volunteers often hesitate between three species: *setosa*, *versicolor* and *virginica*. The team asks you, an ISIMM software-engineering student acting as a junior ML engineer, two questions before anyone trusts a model:

- Q1.** Do the four measurements actually carry information about the species?
- Q2.** How accurate would a simple model be on *flowers it has never seen*, compared with a trivial guess?

You will answer with evidence, using the real Iris data published by R. A. Fisher (1936), which ships with scikit-learn (150 flowers, exactly 50 per species, no missing values). A teammate has already claimed “100% accuracy”; part of your job is to check that claim.

2 Learning outcomes

By the end of the session you will be able to:

- LO1 Predict and verify** the result and shape of NumPy vectorized operations, axis reductions and broadcasting, and explain a broadcasting error.
- LO2 Inspect and summarize** a tabular dataset with Pandas and Matplotlib, and **justify** from evidence which features discriminate the classes.
- LO3 Implement and interpret** a correct evaluation protocol: stratified train/test split, majority-class baseline, k -NN classifier, test accuracy.
- LO4 Diagnose** errors both in a model's predictions (confusion matrix) and in a flawed evaluation protocol.

3 Prerequisites and resources

Knowledge. Python basics (variables, functions, loops, lists/dicts, imports); Chapter 1 vocabulary: feature, target/label, supervised learning, train/test split, baseline, generalization.

Supplied files (course platform):

lab01_starter.ipynb	Notebook to complete (same content as lab01_starter.py, which can be used without Jupyter).
check_env.py	Environment check for Task 1.
flawed_experiment.py	Teammate's script for Task 7 — do not modify before reading it.
requirements.txt	List of required packages.

Software. Python ≥ 3.11 recommended; numpy, pandas, matplotlib, scikit-learn; Jupyter (recommended, not mandatory). Any ordinary laptop is sufficient: no GPU, under 1 GB RAM, everything runs in seconds. No account, API key or paid service is needed.

Pre-lab — do this BEFORE the session (about 15–20 min)

Installing packages over a busy classroom network is slow. At home, run:

```
python -m venv ml-env
source ml-env/bin/activate          # Windows: ml-env\Scripts\activate
pip install -r requirements.txt     # installs the packages listed in the file
python check_env.py                # must end with "Environment OK."
```

Offline / low-resource fallback: once installed, the lab needs no internet (Iris is stored inside scikit-learn). If Jupyter is not available, run `lab01_starter.py` in any editor or with `python -i lab01_starter.py`; write your answers in the comment blocks.

4 Session plan

Phase	Work	Time
Introduction	Scenario + Part 0 prerequisite check	10 min
Setup and guided task	Task 1 (environment) + Task 2 (NumPy)	15 min
Core practical work	Task 3 (Pandas) 12 · Task 4 (plots) 10 · Task 5 (model vs baseline) 18	40 min
Error analysis and transfer	Task 6 (prediction errors) 7 · Task 7 (flawed protocol) 8	15 min
Synthesis	A7 + exit ticket + debrief	10 min
Total		90 min

Times are planning estimates. If you fall behind, keep moving: Tasks 5–7 matter more than polishing plots.

5 Part 0 — Prerequisite check (5 min, in the notebook)

Answer in one line each in the **Part 0** cell. These are not graded; they tell you (and the instructor) what to revise.

- P1.** In the Iris problem, what are the features and what is the target?
- P2.** Why must a model be evaluated on data it did not see during training?
- P3.** What is a baseline, and why do we need one?
- P4.** Is this task supervised or unsupervised? Classification or regression?

6 Tasks

6.1 Task 1 — Verify the environment (guided)

Purpose: confirm that the toolchain works before any analysis.

Estimated time: 5 min **Inputs:** `check_env.py`, first code cells of the notebook

Outcome(s): prerequisite **Answer cell(s):** —

Instructions. (1) Run `python check_env.py` in a terminal. (2) Open `lab01_starter.ipynb`, run the imports cell and the Task 1 cell. (3) Write your names at the top.

Deliverable & self-assessment

Deliverable: Task 1 cell output showing the four package versions.

Check: no `ModuleNotFoundError`; `check_env.py` ends with “Environment OK.” If not, call the instructor now — do not lose 15 minutes alone.

6.2 Task 2 — NumPy: predict, then verify (guided)

Purpose: practise the three NumPy ideas used in every later lab: vectorization, axes, broadcasting.

Estimated time: 10 min **Inputs:** Task 2 cells

Outcome(s): LO1 **Answer cell(s):** A1

Reminder — axes and broadcasting

For a 2-D array of shape (n, p) : `axis=0` aggregates *down the rows* and returns one value per column, shape $(p,)$; `axis=1` aggregates *across the columns*, shape $(n,)$. **Broadcasting rule:** NumPy compares shapes from the *last* dimension backwards; two dimensions are compatible when they are equal or one of them is 1 (missing leading dimensions are treated as 1). Compatible arrays are combined without copying data in a Python loop.

Instructions.

- 2.1 For each print in cell 2.1, **write your predicted output as a comment first**, then run and compare.
- 2.2 In 2.2, predict the *shape* of the column means and of the row means of `M` (shape $(3, 4)$), then compute them with `.mean(axis=...)`.
- 2.3 In 2.3, compute `mu`, the vector of the 4 feature means of the Iris matrix `X_all` (shape $(150, 4)$), and subtract it from every row **without a loop**. Print the shape of the result and verify with `np.allclose` that its column means are ≈ 0 .
- 2.4 Cell 2.4 deliberately fails. Read the error message, explain it, then write a corrected *row-wise* centering that runs.

Progressive hints — open one at a time, only if stuck

Hint 1. In 2.3 you want one mean *per feature*, i.e. per column. Which axis is that?

Hint 2. `mu` has shape $(4,)$ and `X_all` has shape $(150, 4)$: compare the trailing dimensions using the rule above.

Hint 3. For 2.4, the row means have shape $(150,)$. The argument `keepdims=True` of `mean` keeps the reduced axis with length 1.

Deliverable & self-assessment

Deliverable: executed cells 2.1–2.4 with your predictions as comments; answer **A1**.

Check: your centered array has shape $(150, 4)$ and `np.allclose(...)` prints `True`; your A1 explains the error in terms of *shapes*, not just “it doesn’t work”.

6.3 Task 3 — Pandas: load, inspect, summarize

Purpose: look at the data before modelling it.

Estimated time: 12 min **Inputs:** `load_iris(as_frame=True)`

Outcome(s): LO2 **Answer cell(s):** A2

Instructions.

- 3.1 Add a column `species` containing the flower names (from `iris.target_names`). Display `head()`, `shape`, `info()` and the number of flowers per species.
- 3.2 Display summary statistics of the 4 feature columns (`describe()`, rounded to 2 decimals).
- 3.3 Count the flowers with petal length > 5 cm and give their species distribution.
- 3.4 Compute the per-species **mean** and **standard deviation** of the 4 features with `groupby`.

Progressive hints — open one at a time, only if stuck

Hint 1. `dict(enumerate(iris.target_names))` builds the mapping `{0: 'setosa', ...}`; a Series has a `.map()` method.

Hint 2. Row selection with a condition: `df[df["petal length (cm)"] > 5]`; counting categories: `.value_counts()`.

Hint 3. `df.groupby("species")[feature_cols].mean()` — select the feature columns so that target is not averaged.

Deliverable & self-assessment

Deliverable: cells 3.1–3.4 executed; answer **A2**.

Check: shape (150, 6) after adding `species`; 50 flowers per species; your per-species table is 3×4 . In A2 you compare the differences between species *with* the spread inside each species.

6.4 Task 4 — Matplotlib: does the data contain signal?

Purpose: see whether, and where, the species separate.

Estimated time: 10 min **Inputs:** `df` from Task 3

Outcome(s): LO2 **Answer cell(s):** A3

Instructions.

- 4.1 Draw two scatter plots side by side (`plt.subplots(1, 2)`): petal length vs petal width, and sepal length vs sepal width. Use one colour per species, a legend, and axis labels with units.
- 4.2 Draw overlaid histograms of petal length, one per species (`alpha` for transparency), with a legend.
- 4.3 In A3, **predict** which confusions a classifier is likely to make. You will check this prediction in Task 6.

Progressive hints — open one at a time, only if stuck

Hint 1. Loop over groups: `for name, group in df.groupby("species"):` and call `ax.scatter(..., label=name)` inside the loop; then `ax.legend()`.

Hint 2. `fig, axes = plt.subplots(1, 2, figsize=(11, 4))` gives two axes: `axes[0]`, `axes[1]`.

Deliverable & self-assessment

Deliverable: two figures; answer **A3** with approximate values (in cm) of the overlap zone.

Check: a reader can tell which colour is which species without reading your code; your prediction is specific ("X will be confused with Y") rather than vague.

6.5 Task 5 — First model versus a baseline

Purpose: obtain an honest estimate of performance on unseen flowers.

Estimated time: 18 min **Inputs:** `X`, `y = load_iris(return_X_y=True)`

Outcome(s): LO3 **Answer cell(s):** A4

Reminder — Chapter 1 evaluation discipline

Split first, fit only on the training set, measure on the test set. **Accuracy** = $\frac{\text{number of correct predictions}}{\text{number of predictions}}$.

A **majority-class baseline** always predicts the most frequent class of the *training* set; a model is only useful if it clearly beats it. **Stratification** keeps the class proportions of the full dataset in both subsets. ***k*-nearest neighbours** (*k*-NN): to classify a flower, find the *k* training flowers closest to it (Euclidean distance over the 4 features) and take the majority species among them.

Instructions.

- 5.1 Split with `test_size=0.3`, `random_state=RANDOM_STATE (42)` and `stratify=y`. Print the shapes and the class counts of `y_train` and `y_test` (`np.bincount`).
- 5.2 Fit `DummyClassifier(strategy="most_frequent")` on the training set. Print which class it always predicts and its **test** accuracy.
- 5.3 Fit `KNeighborsClassifier(n_neighbors=3)` on the training set. Print its **test** accuracy and the improvement over the baseline in percentage points.

Progressive hints — open one at a time, only if stuck

Hint 1. Look at your training class counts. What does “most frequent” mean when several classes are equally frequent? Check empirically with `np.unique(baseline.predict(X_test))`.

Hint 2. Express each accuracy as a fraction of the test-set size first (e.g. $\frac{30}{45}$); the decimals then follow.

Hint 3. Limitations: how many test flowers are there? By how much does accuracy change if one more flower is misclassified?

Deliverable & self-assessment

Deliverable: cells 5.1–5.3 executed; answer **A4**.

Check: train and test shapes are (105, 4) and (45, 4); no accuracy is computed on training data; A4 explains the baseline value *exactly* (not just “about one third”) and names one limitation.

6.6 Task 6 — Error analysis: which flowers does the model get wrong?

Purpose: go beyond a single number and relate errors to the data.

Estimated time: 7 min **Inputs:** `knn`, `X_test`, `y_test` from Task 5

Outcome(s): LO4 **Answer cell(s):** A5

Reminder — confusion matrix

Entry (*i*, *j*) of `confusion_matrix(y_true, y_pred)` counts the test examples whose true class is *i* and predicted class is *j*. Correct predictions are on the diagonal; every off-diagonal count is an error type.

Instructions.

- 6.1 Display the confusion matrix as a DataFrame with species names on rows (true) and columns (predicted).
- 6.2 List the misclassified test flowers: index, 4 features, true and predicted species.
- 6.3 Re-draw the petal scatter plot (all flowers, light colours) and mark the misclassified test flowers with a large red x.

Progressive hints — open one at a time, only if stuck

Hint 1. `wrong = np.where(y_pred != y_test)[0]` gives the positions of the errors.

Hint 2. `iris.target_names[y_test[wrong]]` converts class numbers into names (NumPy fancy indexing).

Deliverable & self-assessment

Deliverable: confusion matrix, table of errors, annotated plot; answer **A5**.

Check: the sum of the off-diagonal counts equals the number of rows in your error table and is consistent with your accuracy in Task 5.

6.7 Task 7 — Transfer: diagnose a flawed experiment

Purpose: recognise evaluation mistakes in someone else's code.

Estimated time: 8 min **Inputs:** `flawed_experiment.py` (lines marked A–D)

Outcome(s): LO3, LO4 **Answer cell(s):** A6

Instructions. Run `python flawed_experiment.py` (or `%run flawed_experiment.py` in the notebook). It prints a perfect accuracy and recommends deployment. The script contains no Python error.

- 7.1 Read the script and identify at least **three** distinct problems in the evaluation *protocol*. Refer to lines A–D.
- 7.2 For each problem, explain *why* it makes the reported number untrustworthy.
- 7.3 Write the corrected protocol in the Task 7 cell (you may reuse Task 5) and report the corrected result.

Progressive hints — open one at a time, only if stuck

Hint 1. On which data is the model fitted at line B? Does that set contain the test flowers?

Hint 2. On which data is the accuracy measured at line C?

Hint 3. With $k = 1$, what is the nearest training neighbour of a training flower — and at what distance?

Deliverable & self-assessment

Deliverable: corrected code and answer **A6**.

Check: each problem is linked to a line and to a consequence; your corrected accuracy is computed on the test set only and compared with a baseline.

7 Synthesis (A7) and expected deliverables

A7 — Synthesis. In 3–4 sentences addressed to the non-technical herbarium team, answer Q1 and Q2 from the scenario: is there signal in the measurements, how accurate is the model compared with a trivial guess, what kind of error does it make, and what is the main limitation of your estimate?

Deliverable & self-assessment

Submit (one per pair, end of session): `lab01_<name1>_<name2>.ipynb` (or `.py`), executed from top to bottom without errors (*Restart & Run All*), containing: Part 0 answers; code and outputs of Tasks 1–7; written answers **A1–A7**. The exit ticket is answered individually (paper or form, as announced by the instructor).

8 Self-check before submitting

- ☐ The notebook runs from top to bottom after a kernel restart.
- ☐ Every figure has a title, axis labels with units and a legend.
- ☐ The baseline and the k -NN accuracies are both measured on the **test** set and given as fractions and decimals.
- ☐ A4 explains the baseline value exactly; A5 relates each error to the plot.

- ☐ A6 lists at least three protocol problems, each linked to a line of the script.
- ☐ A7 is understandable by someone who has never written Python.

9 Reflection and exit ticket

Reflection (not graded, discuss in your pair): Which step would you now refuse to skip before trusting any model? Which NumPy or Pandas operation do you still need to practise?

Exit ticket — individual, 5 minutes

- E1.** A classmate reports 0.40 test accuracy on a 3-class problem with balanced classes. What can you conclude, and what would you check first?
- E2.** Give one precise reason why accuracy measured on the training set is not a valid estimate of performance on new data.
- E3.** A has shape (20, 3). What is the shape of $A - A.mean(axis=0)$? What happens with $A - A.mean(axis=1)$, and why?

10 Optional advanced challenges

Only after Tasks 1–7 and A7 are complete. Not required for the learning outcomes; they may earn feedback, not core points.

- C1. Baseline from scratch.** Re-implement the majority-class baseline with NumPy only (`np.unique(..., return_counts=True)`) and check that it matches `DummyClassifier`, including on ties.
- C2. Choosing k correctly.** Select $k \in \{1, \dots, 20\}$ using 5-fold stratified cross-validation *on the training set only*, then evaluate the chosen model *once* on the test set. Why is choosing k by looking at test accuracy a mistake? What do you do when several k have the same score?
- C3. How stable is the estimate?** Repeat the split for 10 different values of `random_state` and report the mean, minimum and maximum test accuracy of 3-NN. What does this say about the single number from Task 5?
- C4. Vectorization, measured.** Time a Python loop versus a vectorized NumPy expression on 10^6 numbers. Report your own timings (they depend on your machine).
- C5. All feature pairs.** Use `pd.plotting.scatter_matrix` and argue which pair is the most discriminative.

Data provenance. Iris data as distributed with scikit-learn (`sklearn.datasets.load_iris`). Its documentation states that the data are taken from Fisher (1936), “The use of multiple measurements in taxonomic problems”, *Annals of Eugenics* 7, Part II, 179–188, and that this copy differs from the UCI Machine Learning Repository version in two data points. It is redistributed inside scikit-learn (package licence: BSD-3-Clause) and is used here for non-commercial teaching.